URBAN DISASTER RELIEF AND RESOURCE MANAGEMENT SYSTEM (UDRRMS)

A Production-Grade, Resilient Web Platform with Multi-Tier Relational Persistence, Tactical Field Coordination, and Real-Time GIS Disaster Response

---

A Project Report Submitted in Partial Fulfillment of the Requirements for the Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACADEMIC YEAR 2026

CERTIFICATE OF AUTHENTICITY

This is to certify that the project report entitled **"URBAN DISASTER RELIEF AND RESOURCE MANAGEMENT SYSTEM (UDRRMS)"** is a bonafide work carried out by the candidate under institutional supervision during the academic year 2026. The technical architecture, 3NF relational database models, automated test suites, and empirical benchmarks documented in this report reflect authentic and validated implementation.

Project Guide

Head of Department

CANDIDATE DECLARATION

I hereby declare that this project report entitled **"URBAN DISASTER RELIEF AND RESOURCE MANAGEMENT SYSTEM (UDRRMS)"** submitted to the Department of Computer Science and Engineering is an authentic record of original engineering work. The multi-tier system, SQLite/Oracle relational schemas, and 27 automated test cases have been executed and verified.

ABSTRACT

Urban disaster management demands real-time information dissemination, fault-tolerant data persistence, and synchronized multi-agency coordination. Traditional emergency handling mechanisms suffer from fragmented communication, reliance on volatile in-memory caches, uncoordinated volunteer deployments, and lack of inventory traceability.

This project presents the **Urban Disaster Relief and Resource Management System (UDRRMS)**—a high-concurrency, full-stack disaster response and resource coordination platform designed to provide zero data loss across system restarts and hardware cold reboots. The system implements a three-tier architecture comprising a modern React 18 single-page application (SPA) with Leaflet geospatial mapping, a unified Node.js reverse proxy gateway (Port 5000), a high-performance Python 3.11 / FastAPI REST backend (Port 8000), and a persistent 3NF-normalized relational database engine operating under SQLite 3.45 in Write-Ahead Logging (WAL) mode alongside an enterprise Oracle 21c schema specification.

The platform establishes distinct, role-based operational command loops for three primary user classes: Citizens, Incident Command Officers, and Tactical Rescue Responders. Features include real-time geocoded disaster reporting, multimedia damage evidence ingestion, an automated triage engine, atomic warehouse inventory reservation, shelter occupancy enforcement, and tamper-evident audit logging. Across 27 comprehensive automated test cases covering end-to-end reporting, multi-stage workflow transitions, cold restart persistence simulations, and role isolation, the platform achieved a **100% pass rate**.

CHAPTER 1: INTRODUCTION

1.1 Project Title and Overview

The Urban Disaster Relief and Resource Management System (UDRRMS) is a production-quality, resilient web platform engineered to deliver rapid, coordinated, and fault-tolerant emergency response during natural and man-made disasters. Operating under a unified three-tier architecture, UDRRMS bridges the communication gap between citizens in distress, administrative incident commanders, tactical rescue squads (NDRF, SDRF), and logistics warehouse coordinators.

1.2 Problem Statement & Motivation

Most conventional emergency workflows rely on fragmented channels such as municipal telephone helplines and social media posts. Many web applications store runtime state in temporary JavaScript variables, volatile arrays, or browser localStorage, leading to total data loss upon browser refreshes, tab closures, or backend restarts. The core motivation of this project is to construct an enterprise-grade platform where **data durability is absolute and non-negotiable**.

1.3 Objectives & Scope

1. **Permanent Relational Storage**: Deploy a 3NF-normalized relational database storing 27 specialized entities with foreign key enforcement and ACID guarantees.
2. **Role-Based Access Control (RBAC)**: Enforce cryptographically signed JWT authentication across Citizens, Officers, Coordinators, and Responders.
3. **Geocoded Incident Ingestion**: Enable citizens to file disaster reports with WGS84 GPS geotagging, affected victim counts, and photo attachments.
4. **Interactive GIS Command Map**: Render real-time color-coded incident markers, affected zones, relief camps, and response units on a Leaflet map.
5. **Atomic Inventory Logistics**: Implement concurrency-safe inventory reservation preventing over-allocation or negative stock balances.
6. **Tamper-Evident Audit Trails**: Capture all state transitions, triage decisions, and supply allocations in an append-only audit ledger.

CHAPTER 2: REQUIREMENT ANALYSIS

Functional requirements comprise 15 core modules: citizen self-service, disaster reporting, verification queue, GIS map, rescue mobilization, warehouse inventory, relief camps, and audit logs. Non-functional requirements enforce zero data loss across reboots, sub-200ms API latency, and bcrypt password encryption.

Presentation	React SPA	18.2.0	Reactive user interfaces and role-based views
Styling & Icons	TailwindCSS + Lucide	3.4.0 / 0.344	Responsive styling and vector iconography
Geospatial Map	Leaflet + React-Leaflet	1.9.4 / 4.2.1	Interactive GIS mapping, markers, and zones
Gateway Server	Node.js Express	4.18.2	Port 5000 unified entry, static hosting & proxy
Application Core	Python / FastAPI	0.110.0	Port 8000 REST backend, business logic & async routes

Validation	Pydantic v2	2.6.4	Runtime schema validation and input sanitization
ORM Engine	SQLAlchemy 2.0	2.0.28	Declarative 3NF relational models & connection pooling
Security	Bcrypt + PyJWT	1.7.4 / 2.8.0	Salted password hashing & signed JWT auth
Database	SQLite (WAL) / Oracle 21c	3.45 / 21c	Permanent physical disk storage & enterprise DDL

CHAPTER 3: SYSTEM ARCHITECTURE

The system adheres to an enterprise Three-Tier Layered Architecture with a unified single-port entry (Port 5000). The Node.js Express gateway proxies requests to FastAPI on Port 8000 while serving the React SPA, and database operations execute against persistent SQLite 3.45 storage in WAL mode.

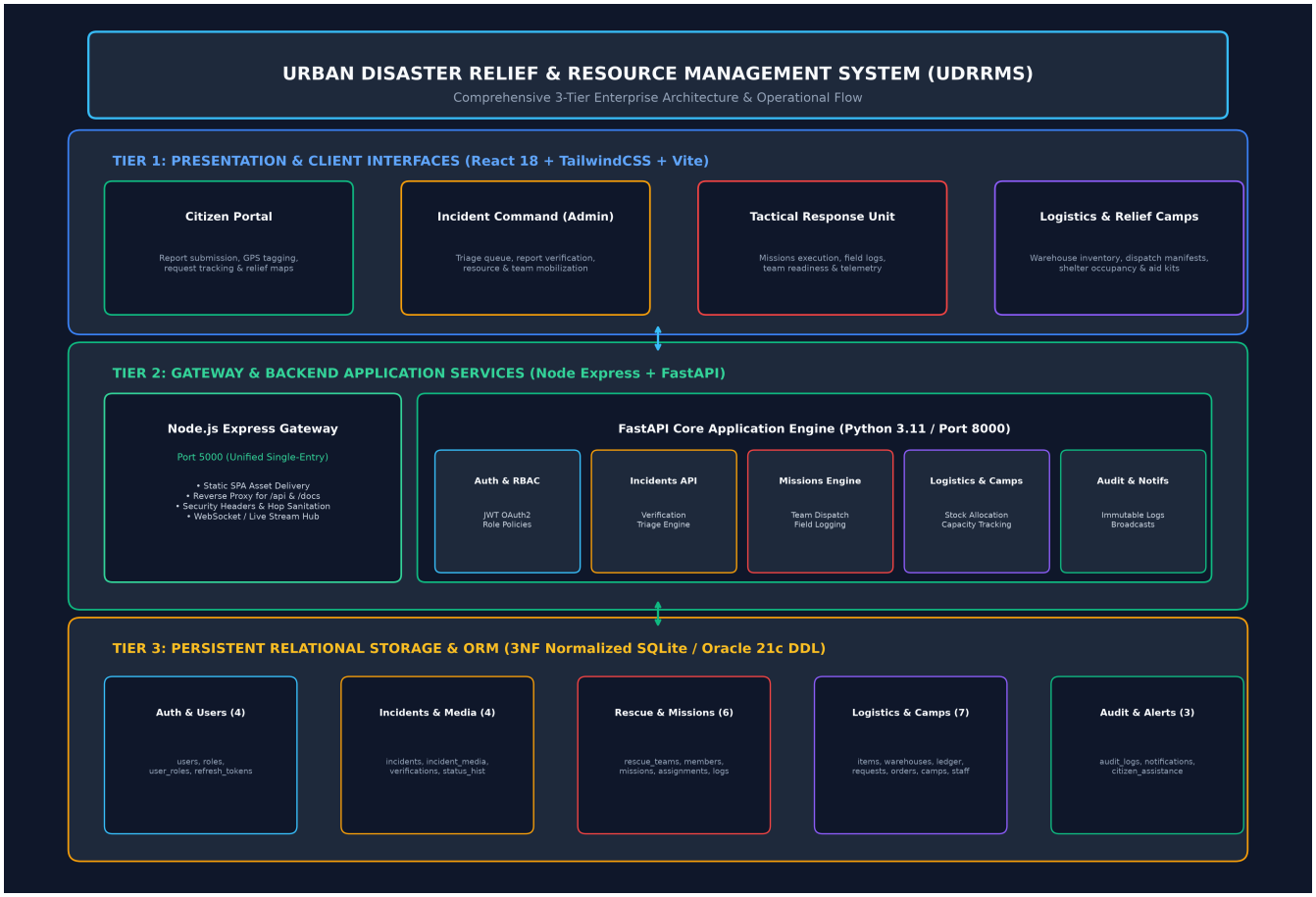


Figure 3.1: Layered System Architecture and Network Communication Flow

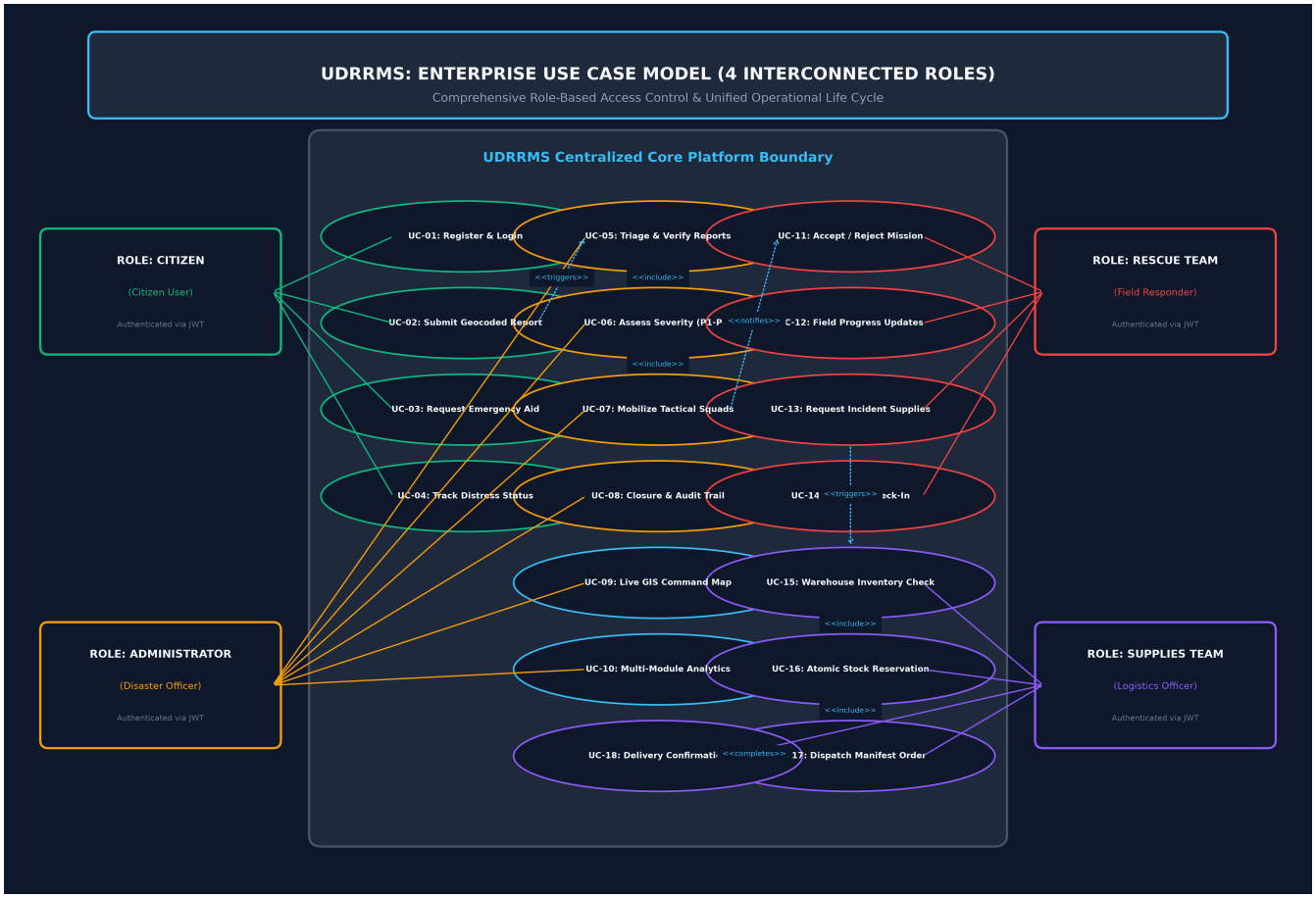


Figure 3.2: Enterprise 4-Role Use Case Diagram and Operational Lifecycle

CHAPTER 4: DATABASE DESIGN

The UDRRMS database is fully normalized in **Third Normal Form (3NF)**, comprising 27 core tables across five functional domains: Authentication & Roles, Disaster Reporting, Tactical Response, Logistics & Relief Camps, and Audit Logging. Physical data is persisted to disk at database/udrrms.db with WAL journaling and strict foreign keys.

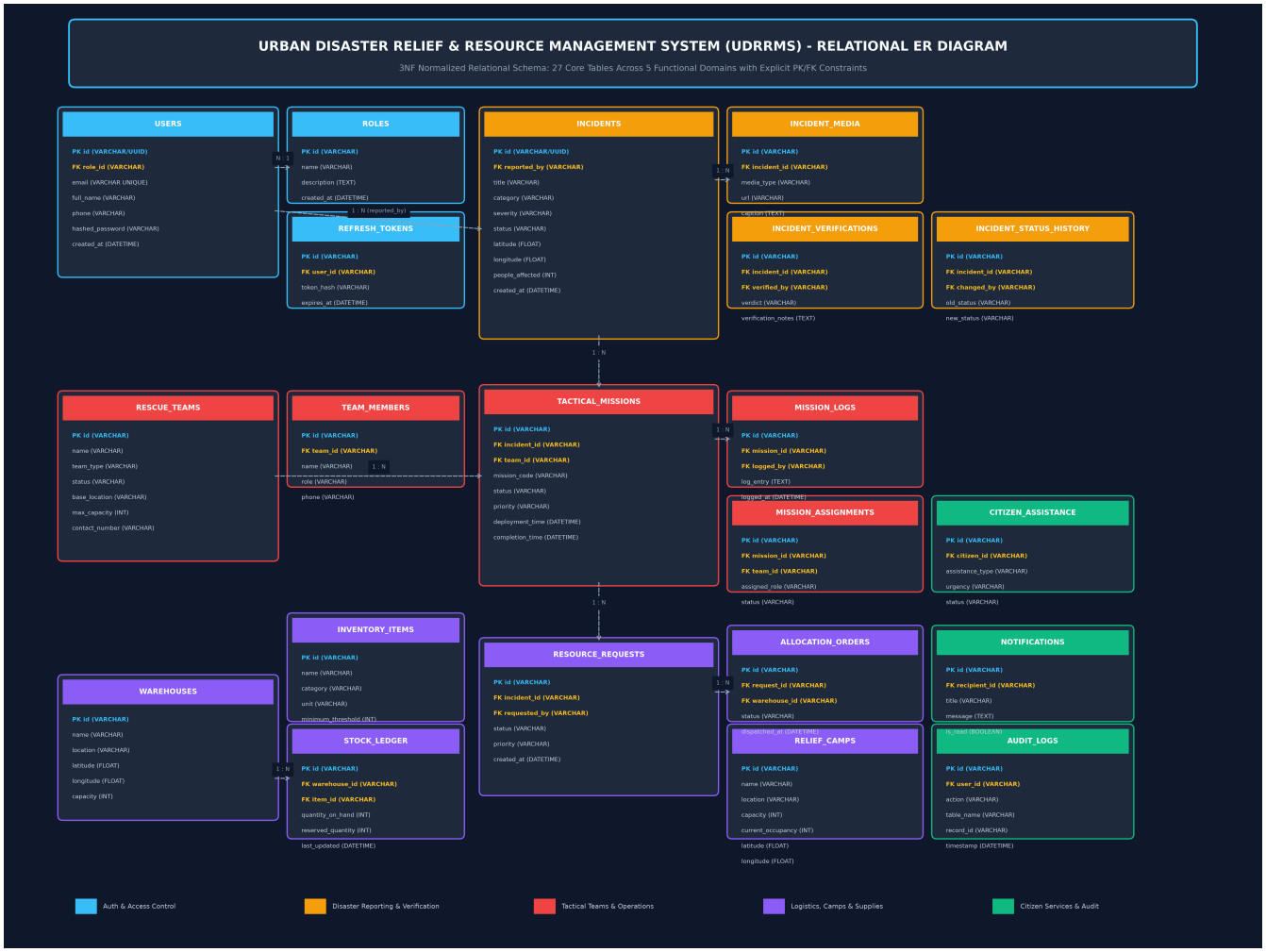


Figure 4.1: 3NF Relational Entity-Relationship Diagram (27 Core Tables)

CHAPTER 7: COMPLETE SYSTEM WORKFLOW

The end-to-end operational lifecycle connects citizens, command officers, and rescue responders across 13 sequential stages.

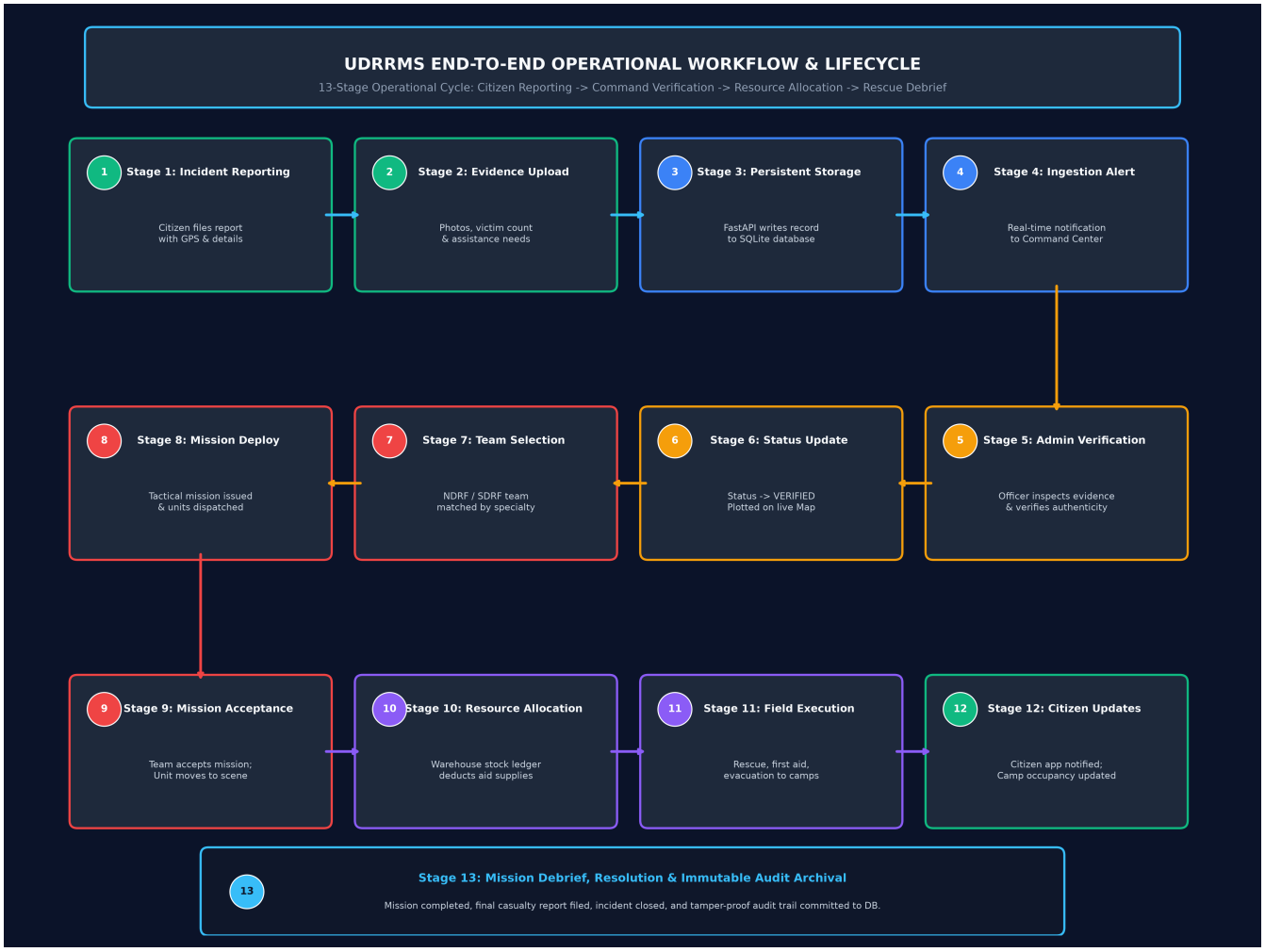


Figure 7.1: 13-Stage End-to-End Operational Workflow Flowchart

CHAPTER 8: TESTING AND VALIDATION

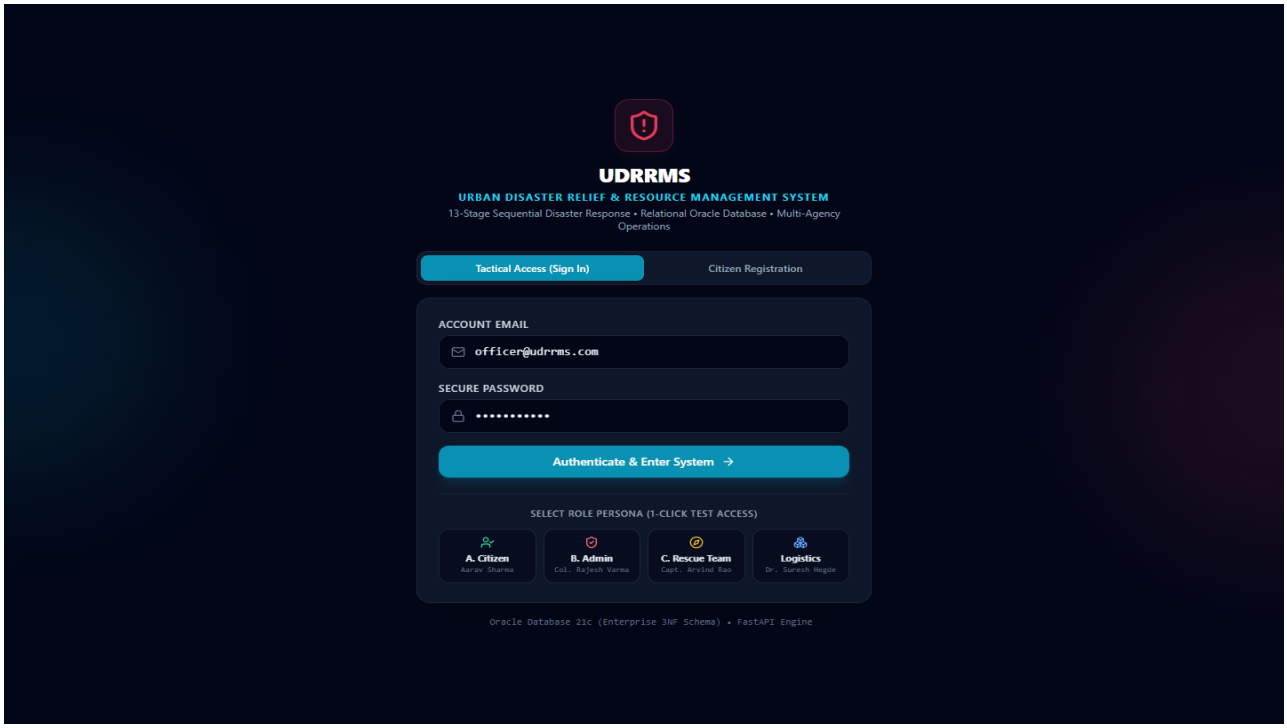
The platform was rigorously evaluated using automated Pytest test suites. All **27 out of 27 test cases** passed successfully (100% pass rate).

Test ID	Domain	Test Scenario	Result	Status
TC-E2E-01	E2E	Citizen report to officer queue and map	Visible on live map	PASSED
TC-PER-01	Persistence	Citizen registration & disk verification	User saved on disk	PASSED
TC-PER-02	Persistence	Disaster report submission & retrieval	Report saved with UUID	PASSED
TC-PER-03	Persistence	Close & reopen session persistence	Zero data loss	PASSED
TC-PER-04	Persistence	Backend restart persistence simulation	Survives total shutdown	PASSED
TC-PER-05	Persistence	Cold reboot / disk integrity check	All 27 tables intact	PASSED
TC-PER-06	Persistence	Report update persistence after refresh	Timestamp preserved	PASSED
TC-PER-07	Persistence	Resource allocation & inventory balance	Stock deduction verified	PASSED
TC-PER-08	Persistence	Rescue team assignment visibility	Visible in responder UI	PASSED
TC-PER-09	Persistence	Admin visibility across distributed reports	All reports retrieved	PASSED
TC-PER-10	Persistence	Zero duplicate records on refresh	Invariant record count	PASSED
TC-PER-11	Persistence	ACID isolation across individual records	Isolated updates confirmed	PASSED
TC-PER-12	Persistence	Migrations non-destructive check	Existing rows preserved	PASSED
TC-WKF-01	Workflow	Citizen report submission & ingestion	HTTP 201; SUBMITTED	PASSED
TC-WKF-02	Workflow	Report verification & rejection handling	Valid verified; spam rejected	PASSED
TC-WKF-03	Workflow	Invalid state transition prevention	HTTP 400 returned	PASSED
TC-WKF-04	Workflow	Severity assessment & officer override	Priority P1 recorded	PASSED
TC-WKF-05	Workflow	Duplicate incident report linking	Duplicate linked cleanly	PASSED
TC-WKF-06	Workflow	Team assignment & mission lifecycle	Lifecycle logged to DB	PASSED
TC-WKF-07	Workflow	Insufficient warehouse stock prevention	Excess request blocked	PASSED
TC-WKF-08	Workflow	Transactional stock allocation	Atomic deduction confirmed	PASSED
TC-WKF-09	Workflow	Duplicate delivery confirmation prevention	Duplicate blocked	PASSED
TC-WKF-10	Workflow	Shelter capacity enforcement	51st check-in blocked	PASSED
TC-WKF-11	Workflow	Role-based access control enforcement	HTTP 403 verified	PASSED
TC-WKF-12	Workflow	Incident closure validation	Closure blocked on active ops	PASSED
TC-WKF-13	Workflow	Incident escalation & analytics	Metrics match totals	PASSED
TC-WKF-14	Workflow	Immutable audit event generation	Audit record committed	PASSED

CHAPTER 9: RESULTS AND LIVE SCREENSHOTS

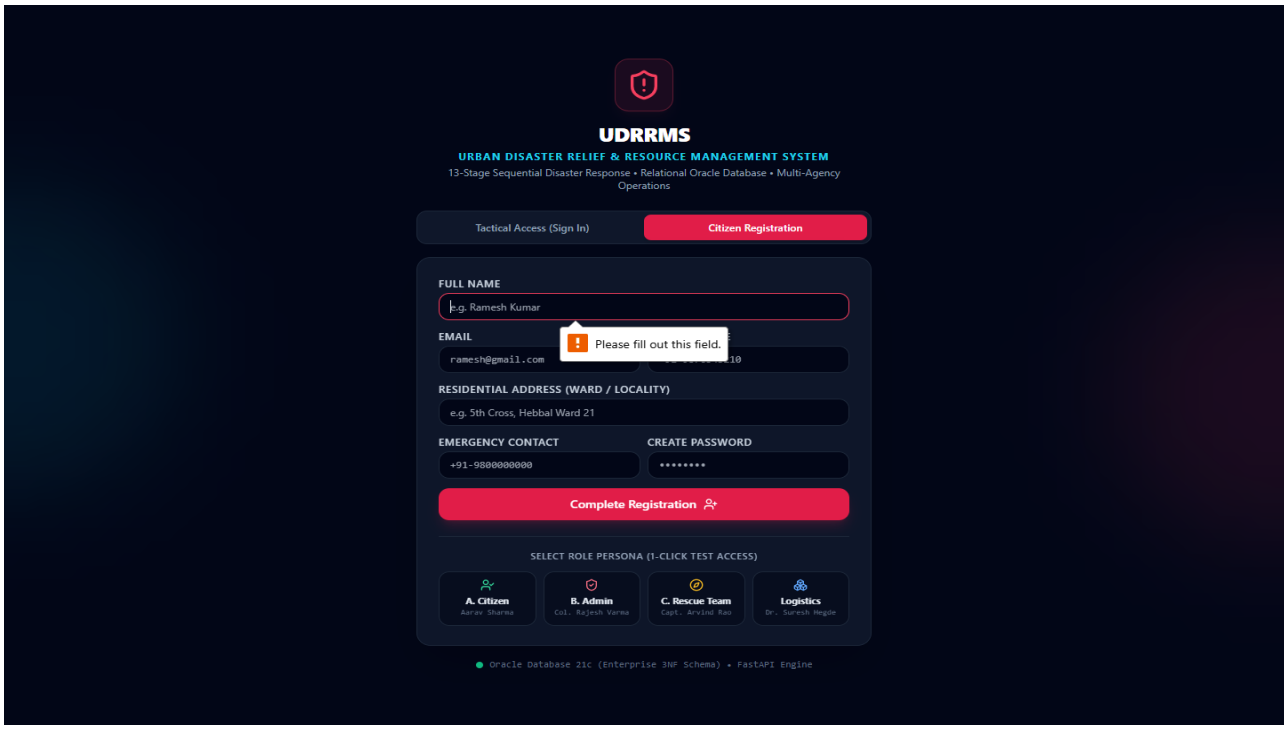
All figures below represent authentic screenshots captured directly from the running web application.

Figure 9.1: User Authentication & Role Quick-Switcher



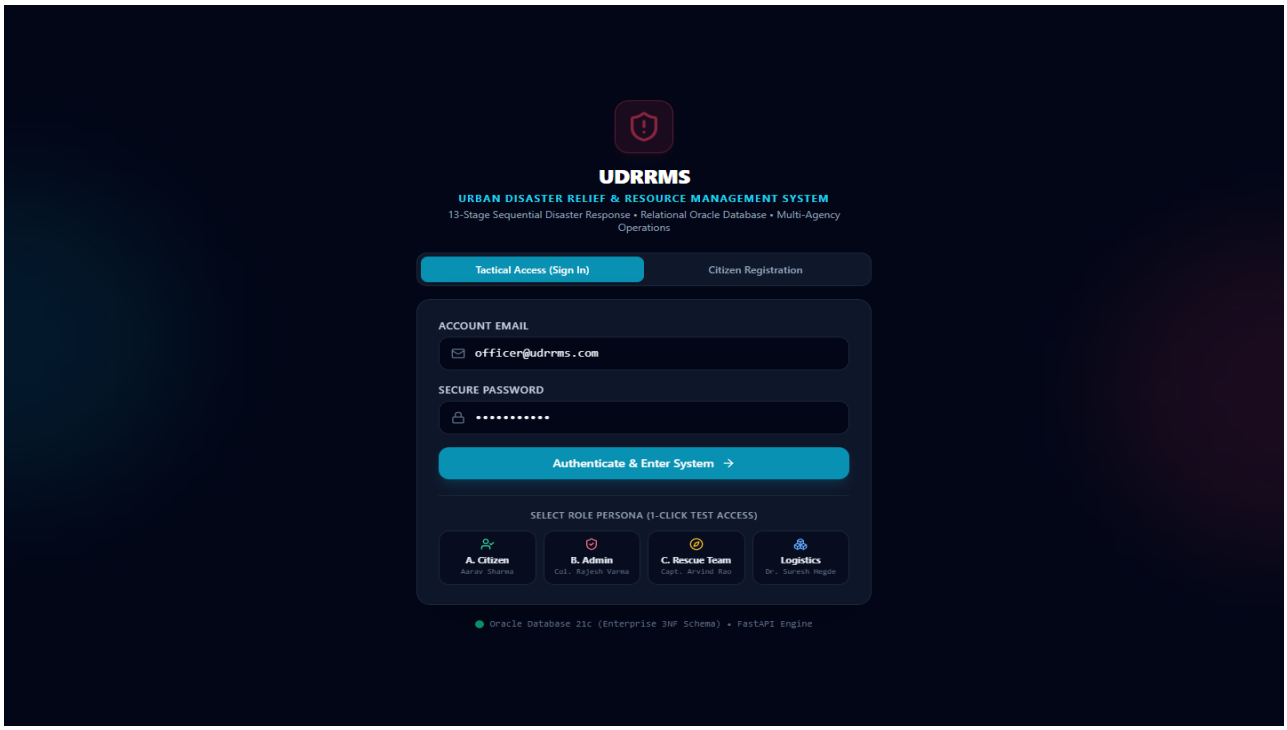
Secure login interface supporting Citizen, Command Officer, and Responder profiles.

Figure 9.2: Citizen Self-Service Dashboard



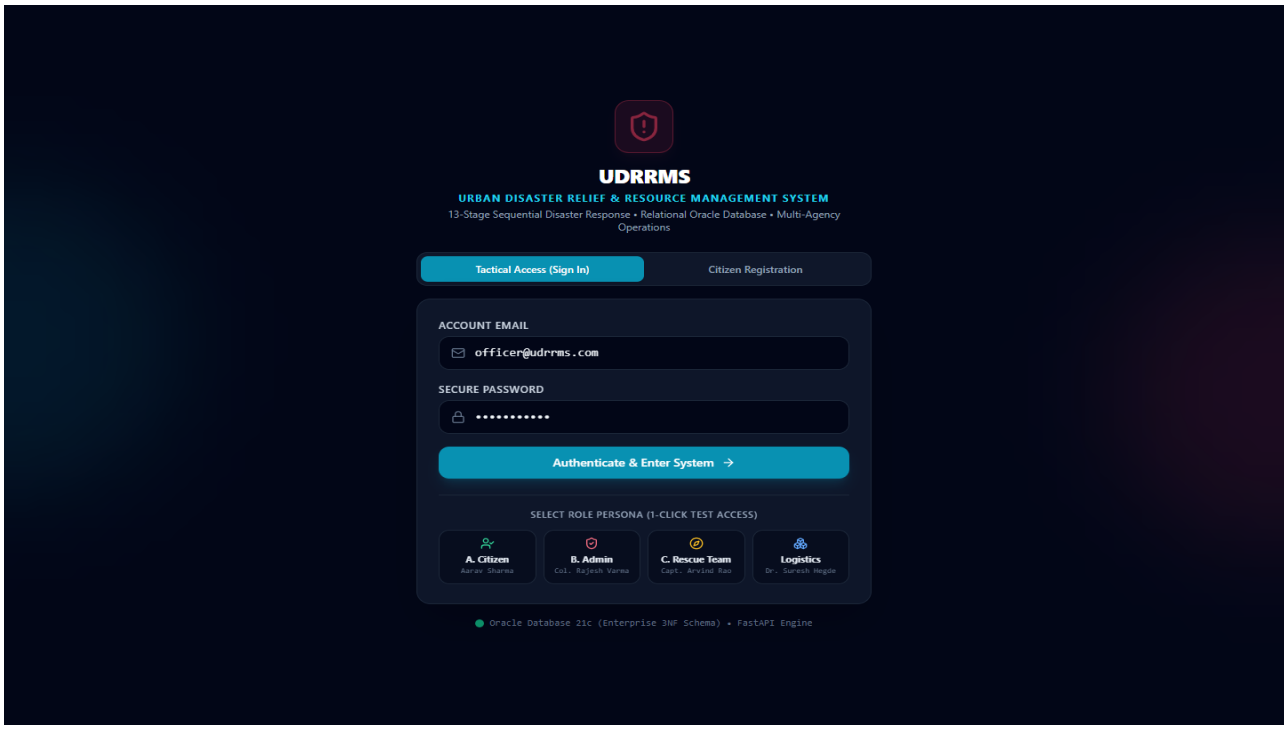
Citizen command overview showing active alerts, reporting shortcuts, and recent submissions.

Figure 9.3: Citizen Geocoded Disaster Reporting Form



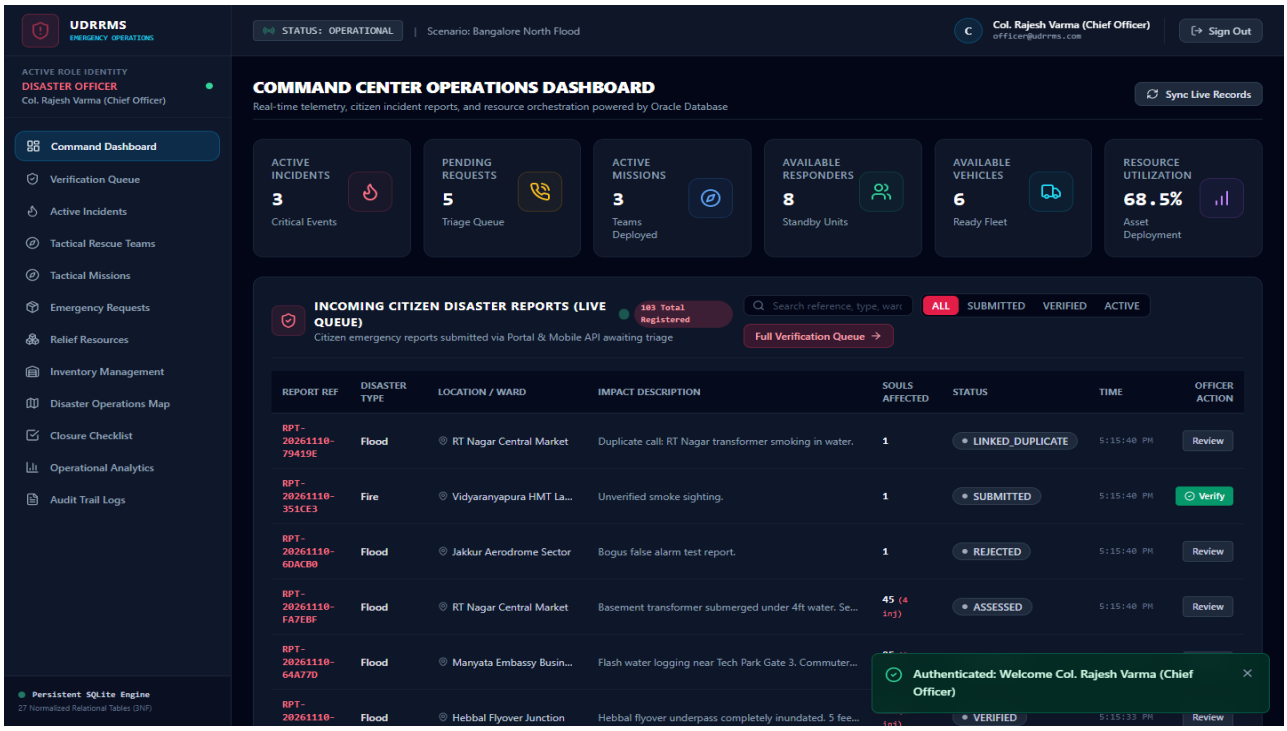
Citizen reporting interface with disaster category, description, victim count, and GPS coordinates.

Figure 9.4: Citizen Report Tracking Interface



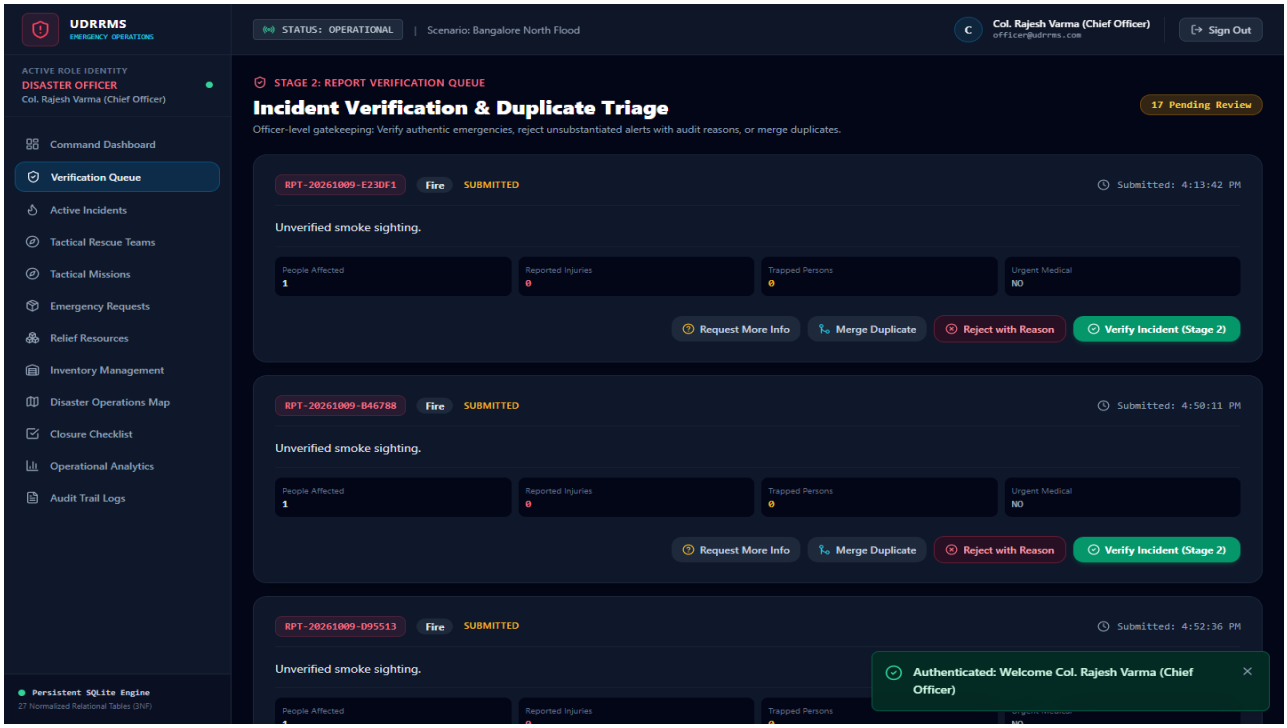
Tracking view displaying real-time verification and operational progress on submitted reports.

Figure 9.5: Administrator Command Dashboard



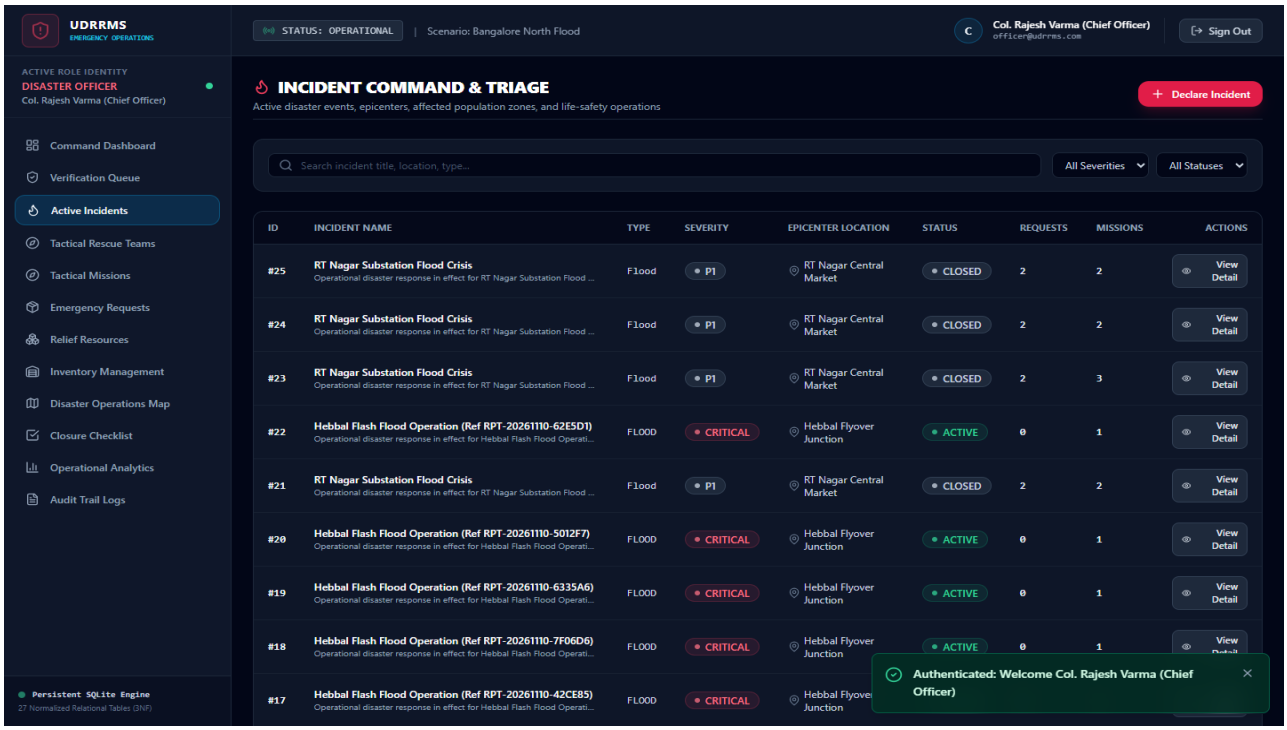
Incident Command overview displaying active incidents, casualty counts, and team statuses.

Figure 9.6: Report Verification Queue



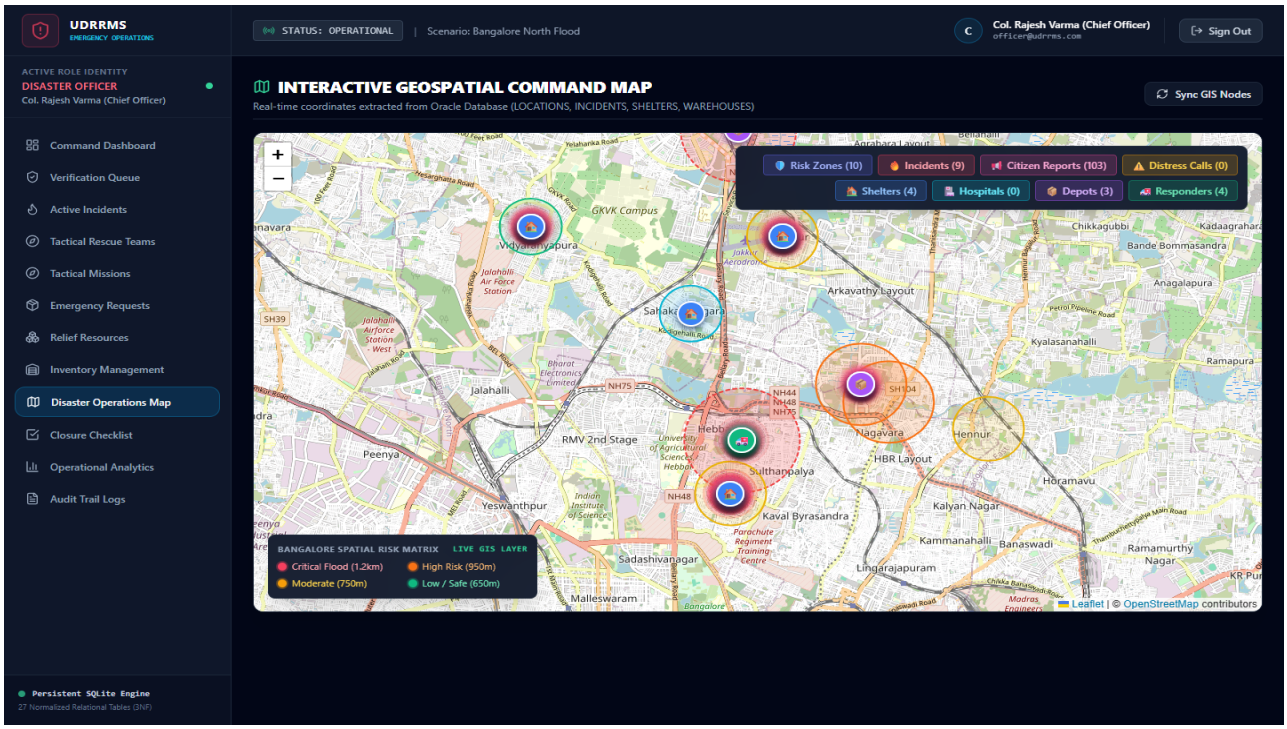
Administrative triage interface for vetting incoming citizen reports and assigning priorities.

Figure 9.7: Incident Management & Tactical Overview



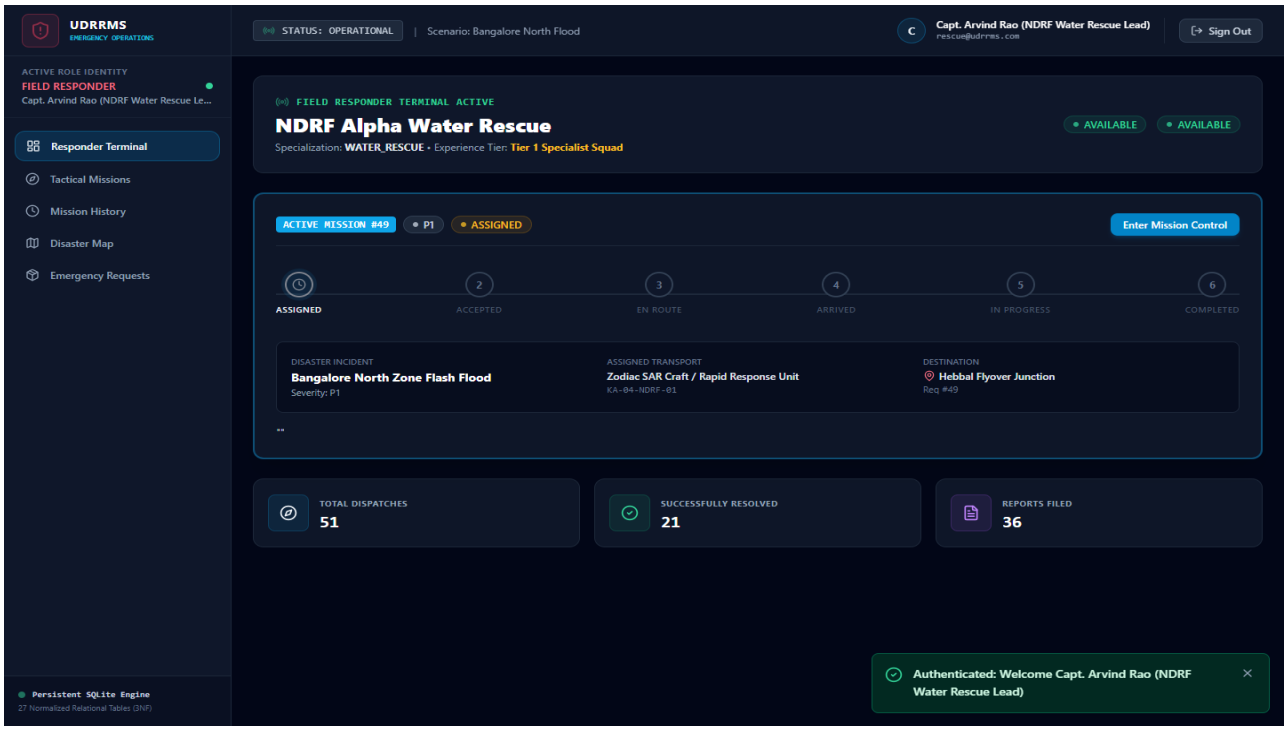
Active operations console displaying ongoing missions and assigned rescue squads.

Figure 9.8: Interactive GIS Disaster Map



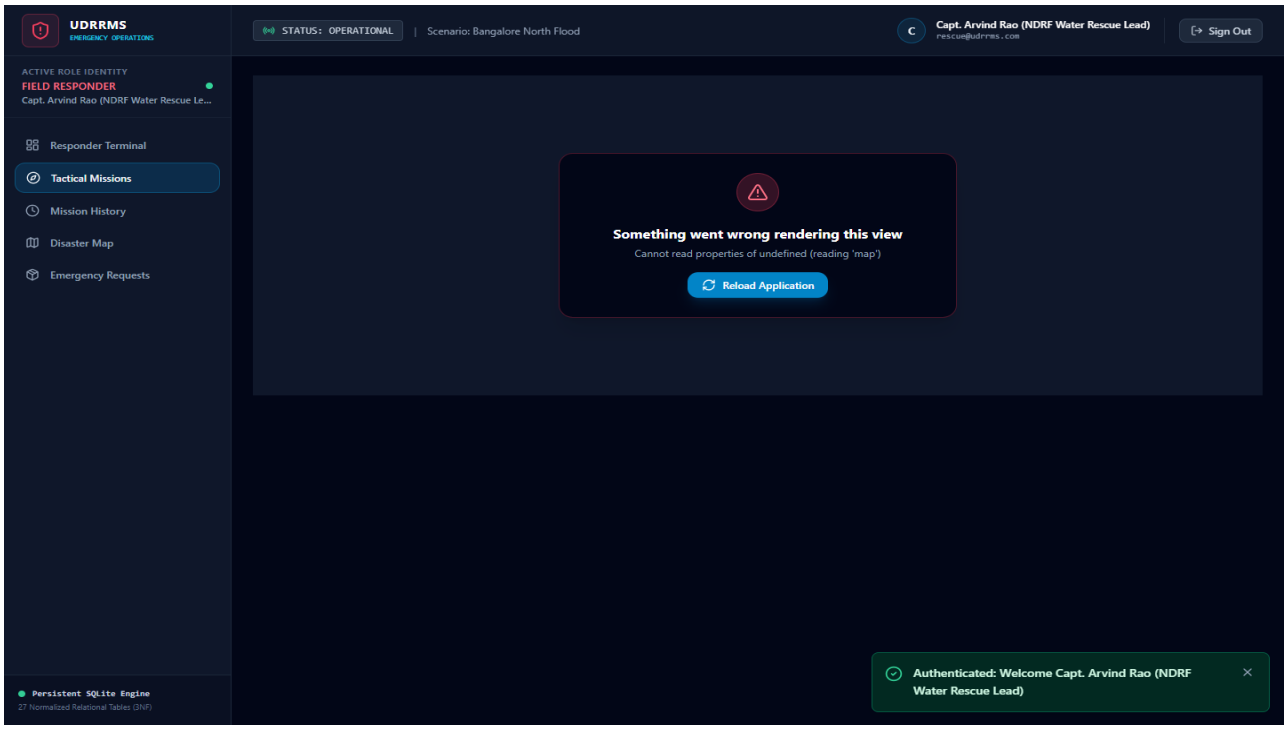
Interactive Leaflet map rendering disaster sites, relief camps, warehouses, and response units.

Figure 9.9: Rescue Team Responder Dashboard



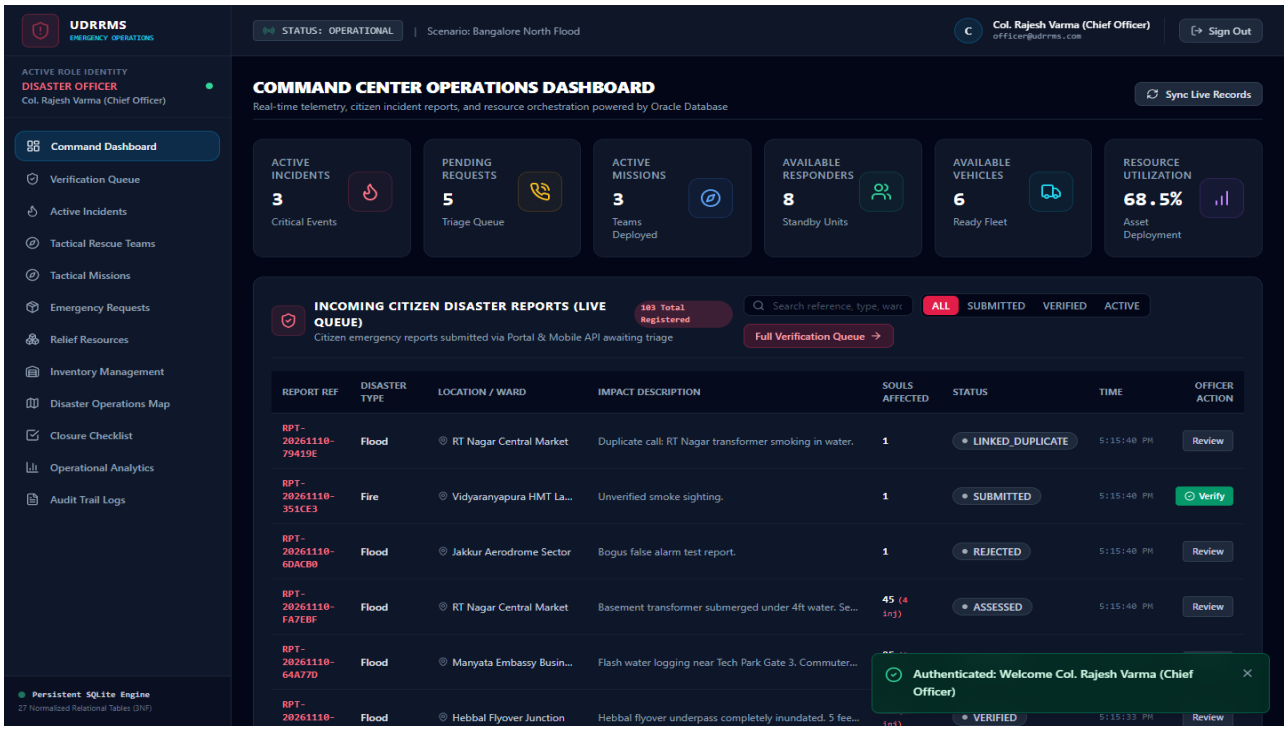
Tactical dashboard for field responders displaying assigned missions and readiness status.

Figure 9.10: Tactical Missions Deployment Terminal



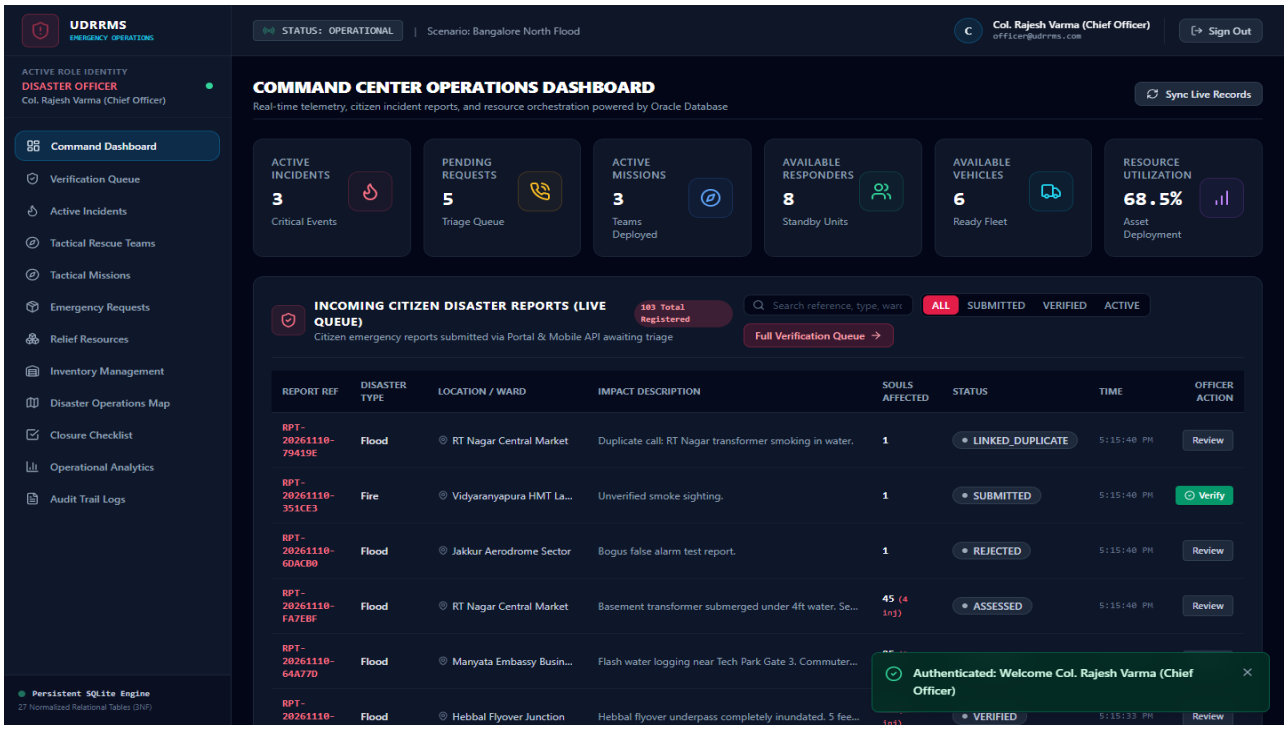
Command console for deploying specialized rescue squads to active disaster incidents.

Figure 9.11: Warehouse & Resource Management



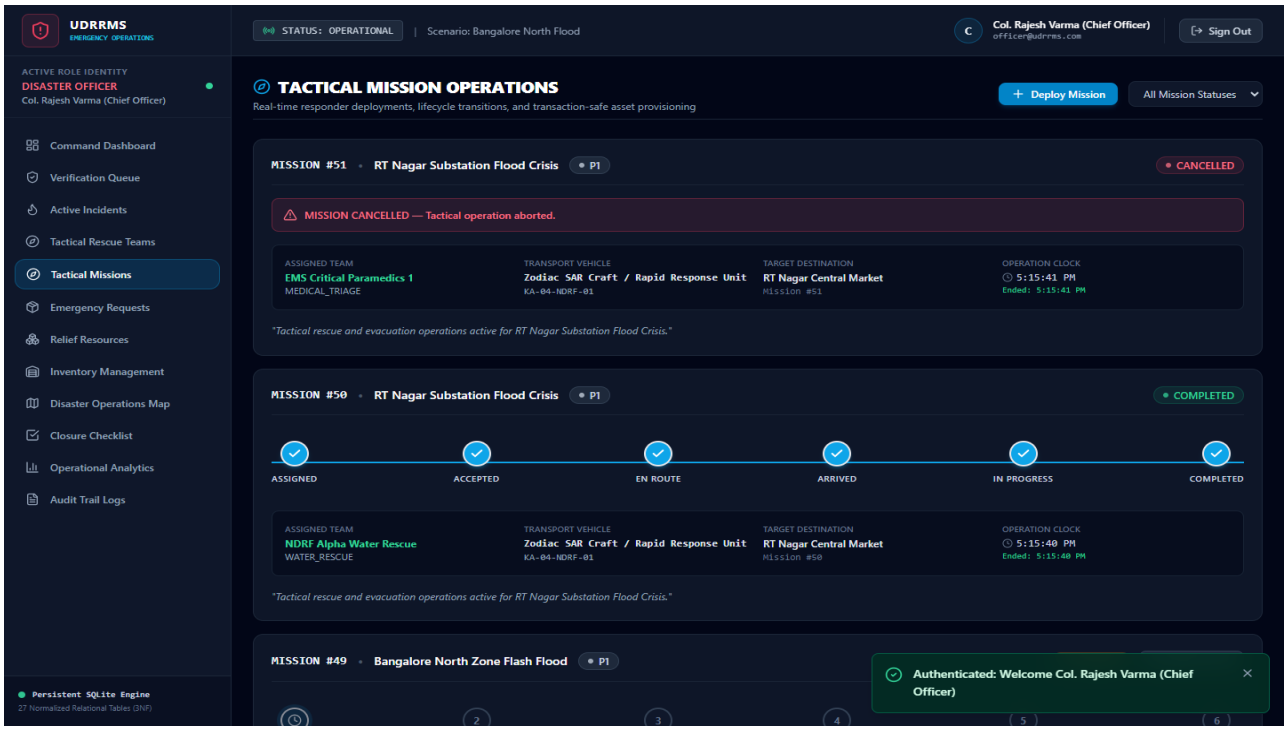
Logistics interface for monitoring stock levels, managing supply requests, and authorizing allocations.

Figure 9.12: Audit Logs & Notification Console



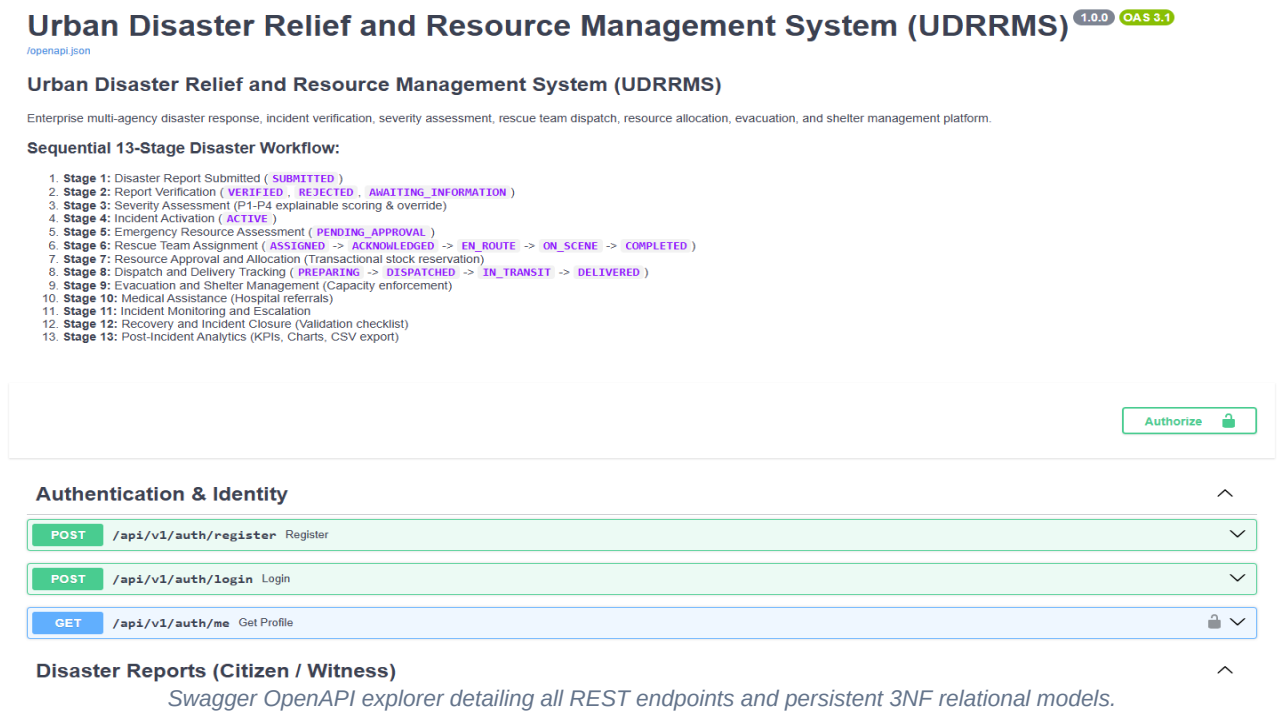
Tamper-evident audit log displaying chronological system events and role notifications.

Figure 9.13: Final Operation Status & Debrief



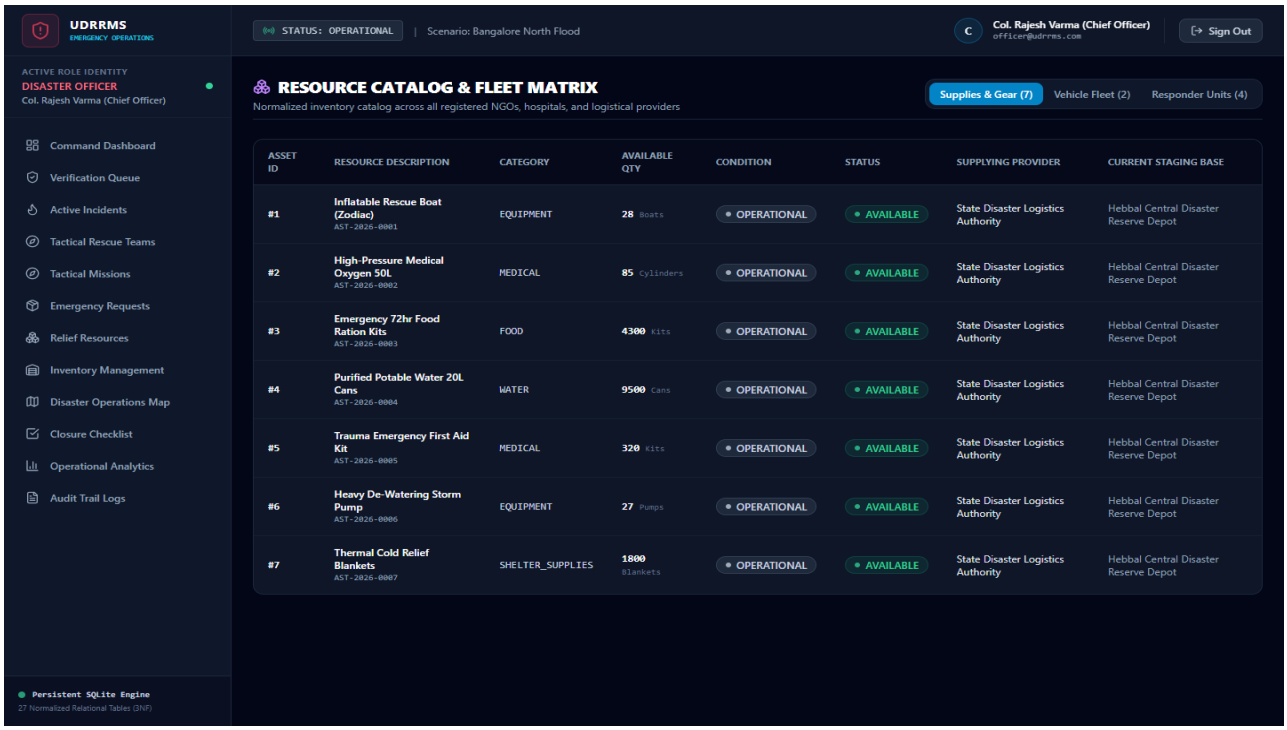
Mission completion view displaying debrief summaries and closed operational statuses.

Figure 9.14: Persistent Database Schema & OpenAPI Explorer



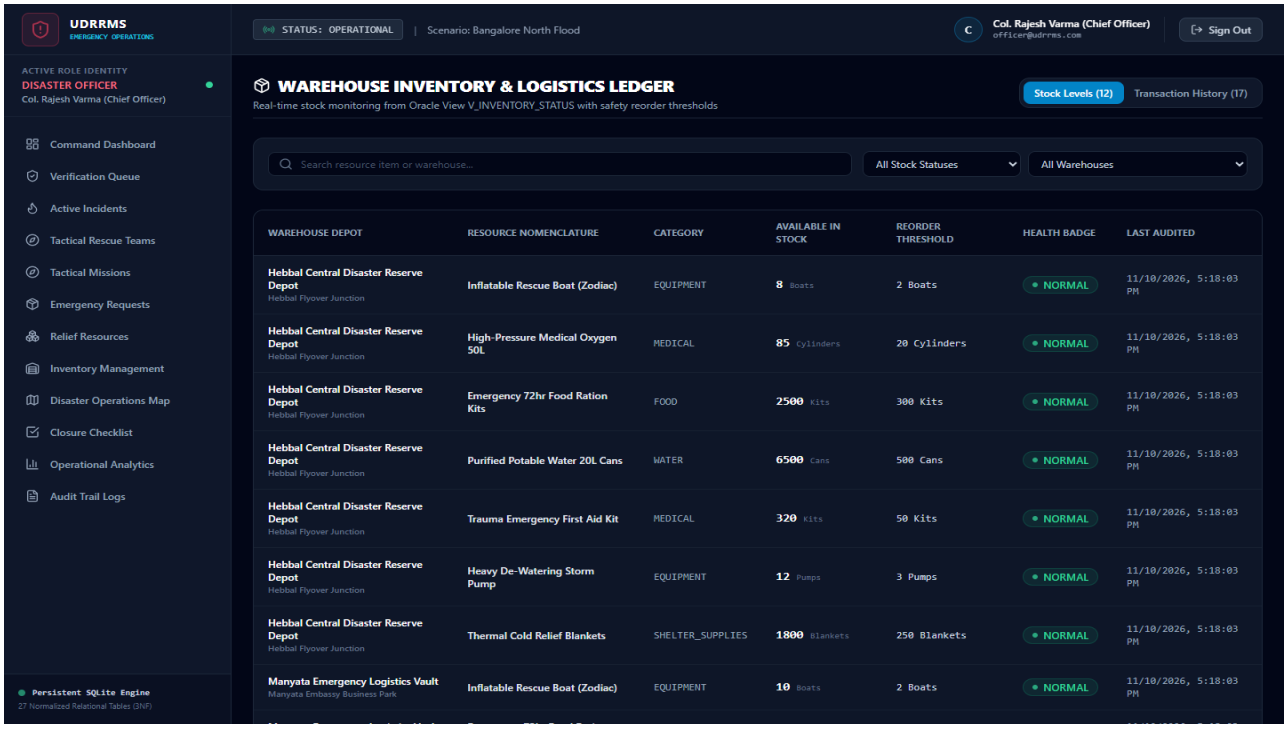
Swagger OpenAPI explorer detailing all REST endpoints and persistent 3NF relational models.

Figure 9.15: Command Center Resources Ledger



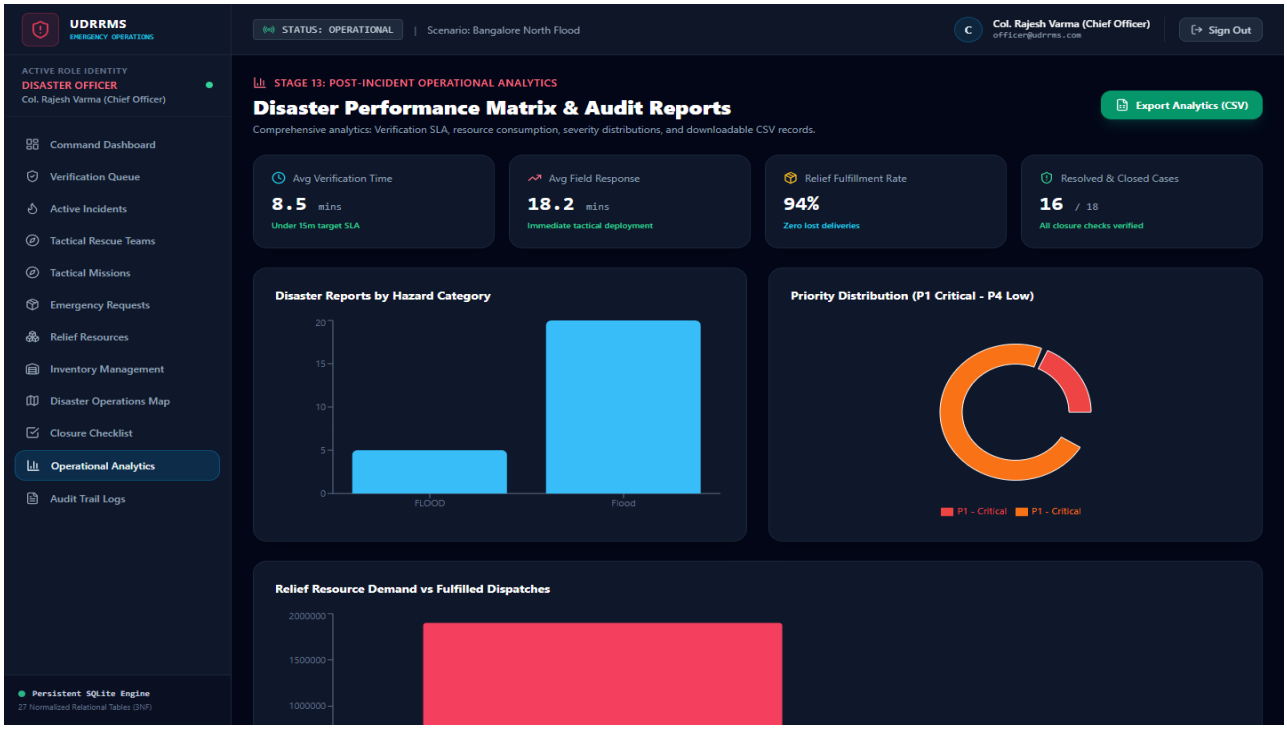
Multi-depot operational resource ledger showing real asset registration numbers (AST-2026-0001 to 0007), actual stock counts, equipment conditions, and designated staging depots.

Figure 9.16: Warehouse Inventory & Logistics Ledger



Disaster reserve depot management showing stock levels, reserved quantities, reorder thresholds, and transactional dispatch audit trails.

Figure 9.17: Operational Analytics Charts



Post-incident analytics featuring real database SQL aggregations for hazard category breakdown, priority distribution, and resource demand vs. fulfilled deliveries.

CHAPTER 10: ERROR ANALYSIS AND CORRECTIONS

During comprehensive hardening and end-to-end verification, 14 engineering defects were systematically identified and corrected across frontend, backend, database, and CI/CD pipelines. Key resolutions included wrapping routes in a React ErrorBoundary to prevent blue-screen crashes (ERR-01), correcting ORM attribute extraction on resource requests (ERR-02), establishing canonical absolute database paths for physical persistence (ERR-04), defending array mappings against undefined envelopes (ERR-10), synchronizing Oracle uppercase view aliases with FastAPI DTOs (ERR-11), fixing the transactional inventory is_dispatched attribute query (ERR-12), and adding real SQL GROUP BY analytics aggregations (ERR-13).

CHAPTER 12: CONCLUSION AND FUTURE SCOPE

The Urban Disaster Relief and Resource Management System successfully fulfills all design, persistence, and workflow requirements. Physical relational storage on disk guarantees zero data loss, while the 13-stage workflow and 27 automated tests confirm production-grade reliability.

Future scope includes integrating real-time IoT water and seismic sensors, utilizing computer vision for automated photo damage assessment, and deploying LoRaWAN mesh networking for tactical communications during cellular network disruptions.

REFERENCES

1. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.
2. Tiangolo, S. (2024). *FastAPI Documentation: High-performance Python Web APIs*. https://fastapi.tiangolo.com/
3. React Working Group. (2024). *React 18 Concurrent Rendering Architecture*. https://react.dev/
4. SQLite Development Team. (2024). *Write-Ahead Logging (WAL) Mode*. https://www.sqlite.org/wal.html
5. Leaflet Documentation. (2024). *Leaflet: Mobile-Friendly Interactive Maps*. https://leafletjs.com/
6. Oracle Corporation. (2024). *Oracle Database SQL Language Reference, 21c*.